

Comparison Delegate in C# with Examples

In this article, I am going to discuss **How to Sort a List of Complex Types using Comparison Delegate in C# with Examples**. Please read our previous article where we discussed [How to Sort a List of the Complex Type in C# with Examples](#).

Generic List<T> Collection Class Sort Method in C#:

The Generic List Collection Class in C# provides the following four Sort Methods.

1. **Sort()**: This method is used to sort the elements in the entire Generic List using the default comparer.
2. **Sort(IComparer<T> comparer)**: This method is used to sort the elements in the entire Generic List using the specified comparer.
3. **Sort(Comparison<T> comparison)**: This method is used to sort the elements in the entire Generic List using the specified System.Comparison.
4. **Sort(int index, int count, IComparer<T> comparer)**: This method is used to sort the elements in a range of elements in a Generic List using the specified comparer.

If you notice, the **Sort(Comparison<T> comparison)** method in the List class, expects the Comparison delegate to be passed as an argument. We know [what is a Delegate](#). A delegate is a function pointer and when we invoke the delegate, the function it points to is going to be executed. So, we need to create one method whose signature must be matched with the signature of the Comparison delegate and then we need to create an instance of the Comparison delegate and to the constructor of the Comparison delegate, we need to pass the method name which we want to execute. Let us understand this Comparison delegate in detail.

Comparison Delegate in C#:

Now, right-click on the Comparison Delegate and then select go to definition, then you will see the following definition of the Comparison Delegate in C#.

```

namespace System
{
    //
    // Summary:
    //     Represents the method that compares two objects of the same type.
    //
    // Parameters:
    //     x:
    //     The first object to compare.
    //
    //     y:
    //     The second object to compare.
    //
    // Type parameters:
    //     T:
    //     The type of the objects to compare.
    //
    // Returns:
    //     A signed integer that indicates the relative values of x and y, as shown in the
    //     following table. Value Meaning Less than 0 x is less than y. 0 x equals y. Greater
    //     than 0 x is greater than y.
    public delegate int Comparison<in T>(T x, T y);
}

```

What is Comparison<T> Delegate in C#?

The Comparison Delegate represents the method that compares two objects of the same type. Here, the parameter x is the first object to compare. The parameter y is the second object to compare. And here T represents the type of objects to be compared. It returns a signed integer that indicates the relative values of x and y, as shown in the following table.

1. Return value greater than ZERO – x is greater than y.
2. Return value less than ZERO – x is less than y
3. The Return value is ZERO – x equals y

Example to Understand Comparison Delegate in C#

Now, let us proceed and see how we can use the Comparison Delegate in C# to compare a collection of complex types. Suppose we have a class called Employee as follows.

```

public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}

```

Also, assume that we have a collection of employees as follows:

```
List<Employee> listEmployees = new List<Employee>
{
    new Employee() { ID = 101, Name = "Pranaya", Gender = "Male", Salary = 5000 },
    new Employee() { ID = 102, Name = "Priyanka", Gender = "Female", Salary = 7000 },
    new Employee() { ID = 103, Name = "Anurag", Gender = "Male", Salary = 5500 },
    new Employee() { ID = 104, Name = "Sambit", Gender = "Male", Salary = 6500 },
    new Employee() { ID = 105, Name = "Hina", Gender = "Female", Salary = 6500 }
};
```

Now, we want to sort the above listEmployees collection using the Sort method of the Generic List<T> Collection Class which takes the Comparison Delegate as a parameter i.e. **Sort(Comparison<T> comparison)**.

How to Use Comparison Delegate in C#?

Step1: Create a function whose signature must be matched with the signature of the Comparison Delegate. This is the method where we need to write the logic to compare 2 Employee objects. We discussed in our delegate article that the delegate signature and the method signature the delegates point to must be the same, otherwise, it will give you compile time error.

```
//The following Method Signature must be the same as Comparison Delegate Signature
//Write the Logic to Sort the Employee
private static int CompareEmployees(Employee e1, Employee e2)
{
    //Sorting the Employees Based on Name
    return e1.Name.CompareTo(e2.Name);
}
```

Step2: Create an instance of Comparison Delegate, and then pass the name of the function created in Step1 as the argument. So, at this point Comparison Delegate is pointing to our function that contains the logic to compare 2 employee objects.

Comparison<Employee> employeeComparer= new Comparison<Employee>(CompareEmployees);

Step3: Pass the delegate instance as an argument to the Sort() method which expects the Comparison Delegate as a parameter as follows.

listEmployees.Sort(employeeComparer);

At this point, listEmployees should be sorted using the logic defined in CompareEmployees() function. The following is the complete Example code:

```
using System;
using System.Collections.Generic;
namespace ComparisonDelegateDemo
{
    public class Program
    {
        public static void Main()
```

```

{
    //Creating a List of Type Employee
    List<Employee> listEmployees = new List<Employee>
    {
        new Employee() { ID = 101, Name = "Pranaya", Gender = "Male", S
5000 },
        new Employee() { ID = 102, Name = "Priyanka", Gender = "Female
7000 },
        new Employee() { ID = 103, Name = "Anurag", Gender = "Male", S
5500 },
        new Employee() { ID = 104, Name = "Sambit", Gender = "Male", S
6500 },
        new Employee() { ID = 105, Name = "Hina", Gender = "Female", S
6500 }
    };
    Console.WriteLine("Employees Before Sorting");
    foreach (Employee employee in listEmployees)
    {
        Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}
{employee.Gender}, Salary = {employee.Salary}");
    }
    //Create an instance of the Comparison Delegate
    Comparison<Employee> employeeComparer = new Comparison<Employee>
(CompareEmployees);
    //Passing Comparison Delegate as an argument to the Sort method
    listEmployees.Sort(employeeComparer);
    Console.WriteLine("\nEmployees After Sorting");
    foreach (Employee employee in listEmployees)
    {
        Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}
{employee.Gender}, Salary = {employee.Salary}");
    }
    Console.ReadKey();
}
//The following Method Signature must be the same as Comparison Delegate
Signature
//Write the Logic to Sort the Employee
private static int CompareEmployees(Employee e1, Employee e2)
{
    //Sorting the Employees Based on Name
    return e1.Name.CompareTo(e2.Name);
}
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}

```

Output:

```
Employees Before Sorting
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
ID = 105, Name = Hina, Gender = Female, Salary = 6500

Employees After Sorting
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 105, Name = Hina, Gender = Female, Salary = 6500
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
```

In this approach, this is what we have done

1. We have created a private function that contains the logic to compare the employees
2. Then created an instance of Comparison delegate, and then passed the name of the private function to the delegate.
3. Finally passed the delegate instance to the Sort() method.

Comparison Delegate with Anonymous Method in C#:

The above code can be simplified using the delegate keyword as shown below which is also known as an anonymous method. This anonymous method signature is also the same as the Comparison Delegate and hence it is also accepted.

```
//Using Comparison Delegate with Anonymous Method
listEmployees.Sort(delegate (Employee e1, Employee e2)
{
    //Sorting the Employees Based on Name
    return e1.Name.CompareTo(e2.Name);
});
```

The Complete Code is given below:

```
using System;
using System.Collections.Generic;
namespace ComparisonDelegateDemo
{
    public class Program
```

```

{
    public static void Main()
    {
        //Creating a List of Type Employee
        List<Employee> listEmployees = new List<Employee>
        {
            new Employee() { ID = 101, Name = "Pranaya", Gender = "Male", Salary = 5000 },
            new Employee() { ID = 102, Name = "Priyanka", Gender = "Female", Salary = 7000 },
            new Employee() { ID = 103, Name = "Anurag", Gender = "Male", Salary = 5500 },
            new Employee() { ID = 104, Name = "Sambit", Gender = "Male", Salary = 6500 },
            new Employee() { ID = 105, Name = "Hina", Gender = "Female", Salary = 6500 }
        };
        Console.WriteLine("Employees Before Sorting");
        foreach (Employee employee in listEmployees)
        {
            Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}, Gender = {employee.Gender}, Salary = {employee.Salary}");
        }
        //Using Comparison Delegate with Anonymous Method
        listEmployees.Sort(delegate (Employee e1, Employee e2)
        {
            //Sorting the Employees Based on Name
            return e1.Name.CompareTo(e2.Name);
        }));
        Console.WriteLine("\nEmployees After Sorting");
        foreach (Employee employee in listEmployees)
        {
            Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}, Gender = {employee.Gender}, Salary = {employee.Salary}");
        }
        Console.ReadKey();
    }
}

public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}

```

Output:

```

Employees Before Sorting
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
ID = 105, Name = Hina, Gender = Female, Salary = 6500

Employees After Sorting
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 105, Name = Hina, Gender = Female, Salary = 6500
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 104, Name = Sambit, Gender = Male, Salary = 6500

```

Comparison Delegate with Lambda Expression in C#

The code that we have written in our previous example using the anonymous method, can be further simplified using the lambda expression as shown below.

```
listEmployees.Sort((e1, e2) => e1.Name.CompareTo(e2.Name));
```

The Complete Code is given below:

```

using System;
using System.Collections.Generic;
namespace ComparisonDelegateDemo
{
    public class Program
    {
        public static void Main()
        {
            //Creating a List of Type Employee
            List<Employee> listEmployees = new List<Employee>
            {
                new Employee() { ID = 101, Name = "Pranaya", Gender = "Male",
5000 },
                new Employee() { ID = 102, Name = "Priyanka", Gender = "Female",
7000 },
                new Employee() { ID = 103, Name = "Anurag", Gender = "Male",
5500 },
                new Employee() { ID = 104, Name = "Sambit", Gender = "Male",
6500 },
                new Employee() { ID = 105, Name = "Hina", Gender = "Female",
6500 }
            };
            Console.WriteLine("Employees Before Sorting");
            foreach (Employee employee in listEmployees)
            {

```

```

        Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}
{employee.Gender}, Salary = {employee.Salary}");
    }
    //Using Lambda Expression
    listEmployees.Sort((e1, e2) => { return e1.Name.CompareTo(e2.Name);
    //listEmployees.Sort((e1, e2) => e1.Name.CompareTo(e2.Name));
    Console.WriteLine("\nEmployees After Sorting");
    foreach (Employee employee in listEmployees)
    {
        Console.WriteLine($"ID = {employee.ID}, Name = {employee.Name}
{employee.Gender}, Salary = {employee.Salary}");
    }
    Console.ReadKey();
}
}
public class Employee
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Gender { get; set; }
    public int Salary { get; set; }
}
}

```

Output:

```

Employees Before Sorting
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 104, Name = Sambit, Gender = Male, Salary = 6500
ID = 105, Name = Hina, Gender = Female, Salary = 6500

Employees After Sorting
ID = 103, Name = Anurag, Gender = Male, Salary = 5500
ID = 105, Name = Hina, Gender = Female, Salary = 6500
ID = 101, Name = Pranaya, Gender = Male, Salary = 5000
ID = 102, Name = Priyanka, Gender = Female, Salary = 7000
ID = 104, Name = Sambit, Gender = Male, Salary = 6500

```

In this article, I am going to discuss [Generic Dictionary<TKey, TValue> Collection Class in C#](#) with Examples. Here, In this article, I try to explain **How to Sort a List of Complex Types in C# using Comparison Delegate** with Examples. I hope this How to Sort a List of the Complex Type using Comparison Delegate in C# with Examples article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.